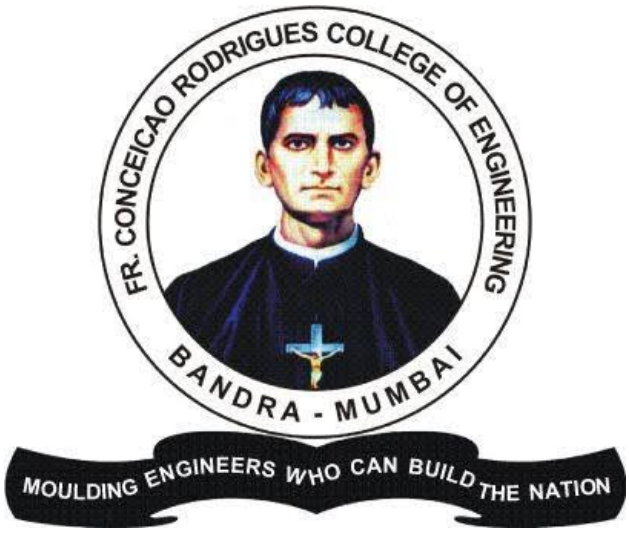


Fr. Conceicao Rodrigues College of Engineering, Bandra - Mumbai

Department of Computer Engineering

A Mini Project Report

Submitted in partial fulfilment of the requirements of the Cloud Computing Laboratory

CampusCare

College Complaint Management System on AWS

Deployed on Amazon EC2 (Nginx + React UI + Flask API) with Cognito authentication, DynamoDB persistence, S3 evidence storage, SNS notifications and gateway VPC endpoints - region us-east-1

Submitted by

Name	Roll No.
Justin Roy	10639
Bhavesh Yadav	10641
Rohit	10629

October 2026

Contents

1. Architecture Diagram 1
2. Abstract 2
3. Problem Statement 3
   - 3.1. Background 3
   - 3.2. Limitations of the existing process 3
   - 3.3. Proposed solution 3
   - 3.4. Objectives 3
   - 3.5. Scope 3
4. Service Inventory 4
   - 4.1. VPC / Network 4
   - 4.2. Compute (EC2) 5
   - 4.3. IAM instance role - CollegeComplaints-EC2-Role 5
   - 4.4. Storage (S3) 6
   - 4.5. Database (DynamoDB) 7
   - 4.6. Identity (Cognito) 7
   - 4.7. Messaging (SNS) 8
   - 4.8. Observability (CloudWatch) 9
   - 4.9. Service summary 9
5. Request Flow (complaint lifecycle) 10
6. Application Walkthrough 11
   - 6.1. Student experience 11
   - 6.2. Administrator experience 13
   - 6.3. Notification delivery 15
7. Conclusion and Future Work 16
8. References 17

Appendix A Resource-ID Quick Reference 18

1. Architecture Diagram

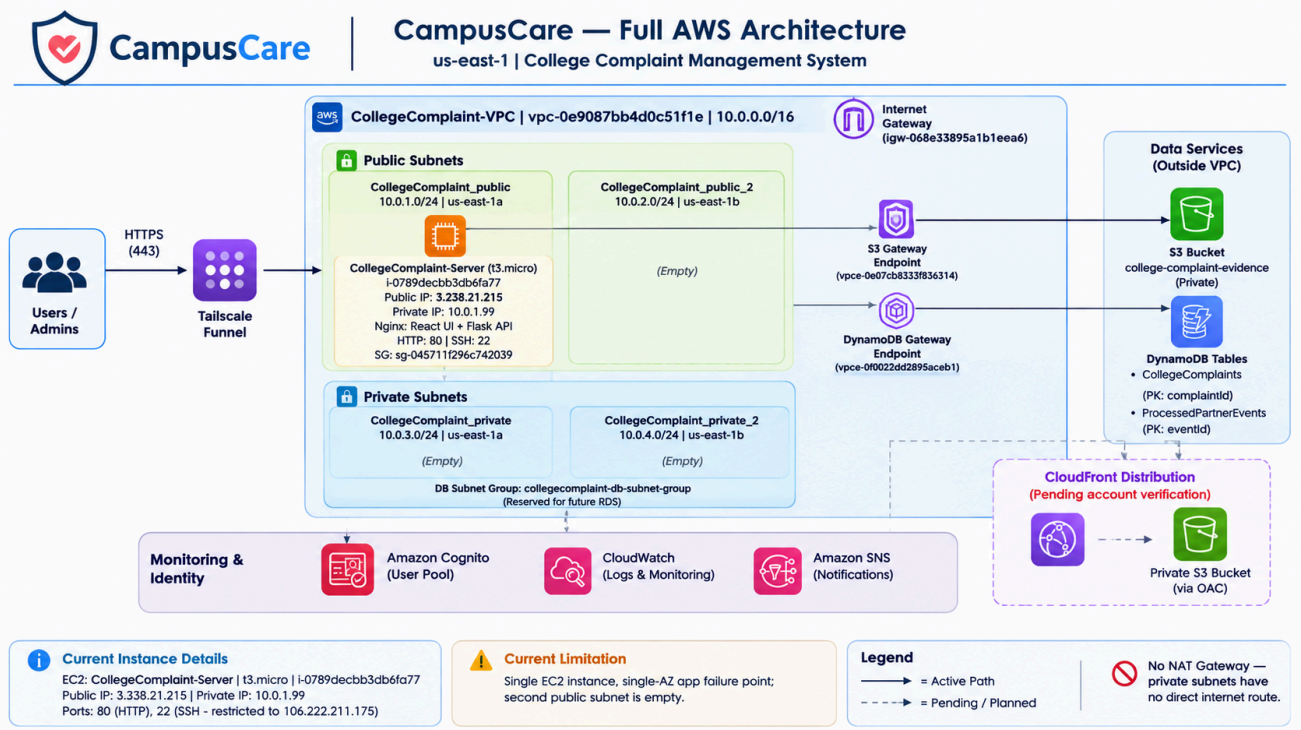


Figure 1: CampusCare - full AWS architecture (us-east-1). Dashed components are planned or pending.

Figure 1 presents the complete deployment as it exists in the AWS account. The system follows a pragmatic single-server baseline: a user reaches the application over HTTPS through **Tailscale Funnel**, which terminates TLS on port 443 of the EC2 instance and forwards requests to **Nginx**. Nginx serves the React single-page application and proxies API calls to the **Flask** backend. The instance is the only compute resource in the project VPC and carries the `CollegeComplaints-EC2-Role` instance profile, so every AWS API call it makes is authorised by temporary role credentials obtained through IMDSv2 - no access keys are stored on the server.

The architecture is organised into the following layers:

- **Ingress.** Tailscale Funnel provides public HTTPS ingress on :443 without a load balancer. The security group permits HTTP/HTTPS from anywhere and SSH only from the administrator's IP address.
- **Compute.** A single `t3.micro` instance (Ubuntu 26.04, 2 vCPU) runs Nginx, the React build and the Flask API in the public subnet.
- **Identity.** An Amazon Cognito user pool authenticates students and administrators through the `CollegeComplaintApp` app client; the backend validates the resulting JWTs against the pool's JWKS endpoint.
- **Data.** Complaint records live in the DynamoDB table `CollegeComplaints` (partition key `complaintId`), while evidence files live in the private S3 bucket `college-complaint-evidence`, one prefix per complaint.
- **Messaging.** Status events are published to the standard SNS topic `CollegeComplaintNotifications`.
- **Observability.** EC2 basic metrics are available in CloudWatch; the application log group `CollegeComplaintApp` exists with one-week retention, and IAM permits the instance to write to it.
- **Network isolation.** The VPC `CollegeComplaint-VPC` (`10.0.0.0/16`) contains two public subnets and two private subnets. Only the first public subnet is used; the private subnets have no internet route and are reserved for a future database tier. S3 and DynamoDB traffic from both route tables is directed through free gateway VPC endpoints, so it never traverses the internet gateway.

2. Abstract

Educational institutions still rely heavily on paper registers, e-mail threads and messaging groups to record student grievances. These channels provide no reliable tracking, no single source of truth and no audit trail, so complaints are frequently lost or resolved without the student ever being informed. **CampusCare** is a cloud-hosted college complaint management system that replaces these ad-hoc channels with a single web application. Students sign in through Amazon Cognito, submit a complaint with supporting evidence and follow its status; administrators review complaints, update their status and trigger notifications.

The application is deployed on a single Amazon EC2 `t3.micro` instance running Ubuntu 26.04, Nginx, a React front end and a Flask API. Complaint metadata is stored in Amazon DynamoDB, evidence files in Amazon S3 and status events are published through Amazon SNS. All service access is granted through a narrowly scoped IAM instance role, so no credentials are stored on the server. The workload runs inside a custom Amazon VPC with a public application subnet, two reserved private subnets and free S3/DynamoDB gateway endpoints. This report documents the deployed architecture, the service inventory, the end-to-end complaint lifecycle and the security posture of the deployment, and it proposes concrete improvements such as CloudWatch alarms, DynamoDB point-in-time recovery, a status GSI and a multi-AZ deployment. The live deployment is available at https://ip-10-0-1-99.taild430d2.ts.net/.

Keywords: cloud computing, Amazon Web Services, Amazon EC2, Amazon Cognito, Amazon DynamoDB, Amazon S3, Amazon SNS, complaint management, VPC, least privilege.

3. Problem Statement

3.1. Background

In most colleges, complaint handling is still a manual, office-centric process. A student who finds a broken laboratory instrument, a hostel maintenance issue or a canteen hygiene problem must visit an office, fill a paper form and then follow up in person. Records are kept in registers or spreadsheets, evidence is attached as printed photographs or forwarded through messaging applications, and status updates are communicated verbally or not at all.

3.2. Limitations of the existing process

- **No unique tracking identifier.** A complaint cannot be referenced reliably after submission, so duplicate complaints and lost forms are common.
- **Scattered evidence.** Photographs, documents and correspondence are spread across paper files, e-mail and chat groups, making retrieval slow and inconsistent.
- **No status visibility.** Students have no way to know whether a complaint has been received, assigned, investigated or resolved.
- **No accountability or audit trail.** There is no timestamped record of who changed what and when.
- **No central reporting.** Administrators cannot easily count, categorise or prioritise complaints across departments.
- **Limited accessibility.** Physical forms are available only during office hours and require a visit to a specific building.

Commercial helpdesk and ticketing products address some of these problems, but they are typically priced per agent, require configuration that is disproportionate to a single college, and do not match the institution's workflow.

3.3. Proposed solution

CampusCare provides a single, low-cost web application for the entire complaint lifecycle. Students authenticate through the application's Cognito user pool, submit a complaint with an optional evidence attachment, and track its progress. Administrators obtain a consolidated list of complaints, inspect the evidence, update the status and notify the student. The application is hosted on AWS so that it can be reached from any browser without the college operating its own server room, and it uses managed services (Cognito, DynamoDB, S3, SNS) so that the operational burden stays small.

3.4. Objectives

1. Provide secure, role-aware access for students and administrators through Amazon Cognito.
2. Allow a student to submit a complaint with a title, description and supporting evidence files.
3. Persist complaint metadata durably and associate each complaint with its evidence prefix.
4. Give administrators a consolidated view of complaints and the ability to update their status.
5. Notify students when the status of their complaint changes.
6. Enforce least privilege by granting the server only the AWS actions it needs, through an IAM instance role.
7. Keep the operating cost close to zero for a pilot deployment.

3.5. Scope

The present deployment covers authentication, complaint submission, evidence storage, complaint listing, status updates and notification. Native mobile applications, online payment, integration with the college ERP, advanced analytics and multi-tenant support are outside the scope of this version and are listed as future work in Section 7.

4. Service Inventory

4.1. VPC / Network

Component	Why it is used	Placement	Configuration and security
VPC	Provides a dedicated address space and routing boundary for the application and its future database tier.	`vpc-0e9087bb4d0c51f1e` (`CollegeComplaint-VPC`), `10.0.0.0/16`, us-east-1.	Default tenancy; all four subnets share NACL `acl-0b9a8611553a0a425`; no VPC peering is used by this project.
Public subnet A	Hosts the EC2 application server, which needs a route to the internet gateway for Tailscale Funnel ingress and OS updates.	`subnet-075715342703e45c8` (`CollegeComplaint_public`), `10.0.1.0/24`, us-east-1a; route table `rtb-0d1f57e36d0516e27`.	Auto-assign public IPv4: No; the instance receives `3.238.21.215`; security group `CollegeComplaint-SG`; shared NACL.
Public subnet B	Second availability-zone slot for a future load-balanced or multi-AZ front end; currently empty.	`subnet-003423eca685068d2` (`CollegeComplaint_public_2`), `10.0.2.0/24`, us-east-1b; same public route table.	No resources; auto-assign public IPv4: No; shared NACL.
Private subnets	Reserved database tier: no internet route, so a future RDS instance cannot be reached from the internet.	`subnet-05e13771ef6f4f787` (`10.0.3.0/24`, us-east-1a) and `subnet-0aa598fe5df874443` (`10.0.4.0/24`, us-east-1b); route table `rtb-092817e4a5469aa36`.	No `0.0.0.0/0` or NAT route; only local and S3/DynamoDB prefix-list routes; shared NACL; both empty.
Internet gateway	The only internet path for the public subnet (Tailscale Funnel and package updates).	`igw-068e33895a1b1eea6` (`CollegeComplaint-igw`) attached to `CollegeComplaint-VPC`.	Attached; the public route table sends `0.0.0.0/0` to the IGW; the private route table has no IGW route.
Route tables	Separate public and private routing domains so the database tier can never reach the internet.	Public `rtb-0d1f57e36d0516e27`; private `rtb-092817e4a5469aa36`; VPC main `rtb-0a77a0d624f300feb`.	Public: local, `0.0.0.0/0` -> IGW, S3 prefix list `pl-63a5400a` -> `vpce-0e07cb8333f836314`, DynamoDB prefix list `pl-02cd2c6b` -> `vpce-0f0022dd2895aceb1`. Private: local + the two prefix-list routes only.
VPC endpoints	Keeps S3 and DynamoDB traffic on the AWS private network and avoids the hourly cost of a NAT gateway.	S3 `vpce-0e07cb8333f836314` (`CC-S3-endpoint`) and DynamoDB `vpce-0f0022dd2895aceb1` (`CC-DDB-endpoint`), gateway type, associated with both route tables.	Status Available; endpoint policy is the default `Allow *`; private DNS is not applicable to gateway endpoints; no hourly charge.
DB subnet group	Pre-places the future RDS tier in the two private subnets so it is multi-AZ ready and never publicly addressable.	`collegecomplaint-db-subnet-group`, private subnets in us-east-1a and us-east-1b.	Status Complete; no RDS instance attached yet; the private route table has no internet path.
Security group	Stateful instance firewall: only application ingress and the administrator's SSH source are allowed.	`sg-045711f296c742039` (`CollegeComplaint-SG`) attached to `i-0789decbb3db6fa77` in the public subnet.	Inbound `80/tcp` and `443/tcp` from `0.0.0.0/0`, `22/tcp` from `106.222.211.175/32`; outbound default all; no IPv6 rules.

Security group rules (CollegeComplaint-SG):

Direction	Port/Proto	Source/Dest	Notes
Inbound	80 / TCP	0.0.0.0/0	HTTP (likely redirect)
Inbound	443 / TCP	0.0.0.0/0	HTTPS app ingress
Inbound	22 / TCP	106.222.211.175/32	SSH admin access - restricted to the admin IP (tightened Oct 29)
Outbound	All / All	0.0.0.0/0	Default allow-all

Design observations

- Two-tier public/private design - the public subnets hold the app server; the private tier (10.0.3.0/24 us-east-1a, 10.0.4.0/24 us-east-1b) is reserved for databases and has no internet path in either direction. S3 and DynamoDB traffic rides free gateway VPC endpoints via prefix-list routes in both route tables - no NAT gateway needed.

- No IPv6, no Elastic IP (public IP is auto-assigned), no NAT gateway and no network-ACL customization (default NACL inherited).
- SSH (22) is now restricted to the admin IP (106.222.211.175/32, tightened Oct 29); user ingress rides Tailscale Funnel, so the SG stays minimal (see the security group rules above).

4.2. Compute (EC2)

Component	Why it is used	Placement	Configuration and security
Instance	Runs the complete application stack (Nginx, React build, Flask API and Tailscale) on one host; a `t3.micro` is sufficient for a pilot and avoids managed-compute overhead.	`i-0789decbb3db6fa77` (`CollegeComplaint-Server`), public subnet `subnet-075715342703e45c8` (`10.0.1.0/24`, us-east-1a); private `10.0.1.99`, public `3.238.21.215`.	Running; status checks 3/3; `t3.micro` (2 vCPU, 1 GiB), credit mode unlimited; default tenancy; no Auto Scaling group or launch template; termination and stop protection disabled.
Base OS	Ubuntu 26.04 LTS provides a supported, security-patched base for Nginx, Python and Tailscale.	Root EBS volume in us-east-1a.	AMI `ami-0d97d7f07b25dd693` (`ubuntu-resolute-26.04-amd64-server-20261016`); key pair `CollegeComplaint`; boot mode UEFI.
Root volume	Holds the OS, application code and Nginx build; 8 GiB is adequate for the pilot.	`vol-0d5b3d830cde5914a`, us-east-1a, attached as `/dev/sda1`.	`gp3`, 8 GiB, 3000 IOPS, 125 MB/s; not encrypted; status check Okay.
Network	The public address makes the instance reachable for Tailscale Funnel and SSH administration; Tailscale terminates TLS.	Elastic network interface in the public subnet; no Elastic IP, no IPv6.	Security group `CollegeComplaint-SG`: `80/443` from `0.0.0.0/0`, `22` from the admin `/32`; outbound all; public IPv4 is auto-assigned.
IAM instance profile	Supplies short-lived AWS credentials to the application without storing access keys on disk.	Attached to the instance; role `CollegeComplaints-EC2-Role`.	IMDSv2 Required; maximum session 1 hour; trust policy allows `ec2.amazonaws.com` to assume the role; four scoped policies (see @sec-iam).
Runtime stack	Serves the SPA and proxies API calls on one host, keeping the deployment simple.	Same instance: Tailscale Funnel `:443` -> Nginx `:80` -> Flask `127.0.0.1:5000`.	Flask `debug=False`; systemd unit `college-complaint.service` enabled; Nginx site `college-complaint`; SPA fallback; `/api/` proxied to loopback only.
Monitoring	Basic metrics give free visibility into CPU, network and status checks.	CloudWatch us-east-1, namespace `AWS/EC2`.	Detailed monitoring disabled (5-minute basic metrics only); no project alarms or dashboard.

Life-cycle notes: normal lifecycle (not Spot), a classic console-managed instance (no launch template, no Auto Scaling).

4.3. IAM instance role - CollegeComplaints-EC2-Role

The instance assumes `CollegeComplaints-EC2-Role` through its instance profile. The trust policy allows only `ec2.amazonaws.com` to call `sts:AssumeRole`; the maximum session duration is one hour, and IMDSv2 is required, so the SDK obtains temporary credentials from the instance metadata service rather than from a stored access key. Four policies scope the role to the four services the application actually calls:

Component	Why it is used	Placement	Configuration and security
S3 (inline)	`CollegeComplaints-S3-Policy` lets the Flask API upload complaint evidence and read it back for administrators.	Attached to the instance role; applies to S3 calls from the EC2 instance.	Allows `s3:PutObject` and `s3:GetObject` on `arn:aws:s3:::college-complaint-evidence/*`; no bucket-level, list or delete permissions.
DynamoDB (managed)	`CollegeComplaints-DynamoDB-Policy` creates, reads, lists and updates complaint records.	Attached to the instance role; table in us-east-1.	Allows `dynamodb:PutItem`, `GetItem`, `Scan` and `UpdateItem` on `arn:aws:dynamodb:us-east-1:*:table/CollegeComplaints`; no `DeleteItem`, `Query` or index actions.

Component	Why it is used	Placement	Configuration and security
SNS (inline)	`CollegeComplaints-SNS-Policy` publishes status-change events for fan-out to the e-mail and SQS subscribers.	Attached to the instance role; topic in us-east-1.	Allows `sns:Publish` on `arn:aws:sns:us-east-1:547846135503:CollegeComplaint-Notifications` only; no `Subscribe`, `SetTopicAttributes` or delete permissions.
CloudWatch (inline)	`CollegeComplaints-CloudWatch-Policy` writes application logs to the dedicated log group.	Attached to the instance role; log group in us-east-1.	Allows `logs:CreateLogStream`, `logs:PutLogEvents` and `logs:DescribeLogStreams` on `arn:aws:logs:us-east-1:547846135503:log-group:CollegeComplaintApp:*`; no `CreateLogGroup` or `DeleteLogGroup`.

None of the policies grants `*:*`. The only wildcard in a resource ARN is the account segment of the DynamoDB table (`us-east-1:*:table/CollegeComplaints`), which still restricts access to the named table in this region. The role has no `iam:*`, no `ec2:*` and no access to any other S3 bucket.

4.4. Storage (S3)

Component	Why it is used	Placement	Configuration and security
Bucket	Stores complaint evidence (photographs, PDFs and other attachments) outside the EC2 disk, so evidence survives instance replacement and is never served publicly.	`college-complaint-evidence` in us-east-1; reached through the S3 gateway endpoint `vpce-0e07cb8333f836314`.	Block all public access ON; Object Ownership bucket-owner-enforced (ACLs disabled); no bucket policy; default encryption SSE-S3 with bucket key enabled; SSE-C blocked.
Prefix layout	One prefix per complaint maps each evidence object to its DynamoDB item and simplifies retrieval and cleanup.	`complaints/{complaintId}/`; a separate `test/` sandbox is also present.	No public listing; the EC2 role is the only principal with object access; no presigned URLs are used.
Access path	The browser never talks to S3 directly, so uploads and downloads stay behind the authenticated API.	EC2 -> S3 gateway endpoint -> S3 (private AWS network).	Server-mediated `PutObject` and `GetObject`; no CORS configuration; no bucket policy; no public ACLs.
Versioning and recovery	Versioning would protect against overwritten or deleted evidence, but it is disabled in the pilot.	Bucket-level setting in us-east-1.	Versioning Disabled; MFA delete Disabled; Object Lock Disabled; no replication or backup rule.
Audit and events	Access logging and event notifications would support investigations and downstream processing; neither is enabled.	Bucket-level setting in us-east-1.	Server access logging Disabled; EventBridge notifications Off; event notifications 0; CloudTrail data events not configured.
Lifecycle	No lifecycle rule is configured, so objects remain in standard storage until deleted manually.	Bucket-level setting in us-east-1.	No tags; no lifecycle or Intelligent-Tiering archive configuration; standard storage class.

Design observations

- ACL/permission hygiene is good: the bucket is fully private; the only principal able to read/write is the server role - the client never touches S3 directly (browser-direct upload would need presigned URLs; the current flow is server-mediated).
- One folder per complaint implies the complaint record carries its own UUID identifier, mapping evidence in S3 to the DynamoDB item.

4.5. Database (DynamoDB)

Component	Why it is used	Placement	Configuration and security
Table	Stores complaint metadata - owner, title, description, status, evidence prefix and timestamps - as a schemaless item keyed by the complaint UUID.	`CollegeComplaints` in us-east-1 (`arn:aws:dynamodb:us-east-1:547846135503:table/CollegeComplaints`); reached through the DynamoDB gateway endpoint `vpce-0f0022dd2895aceb1`.	Active; on-demand capacity; 7 items, 2.1 KB table size, 297 B average item; partition key `complaintId` (String), no sort key; no GSI/LSI.
Access pattern	Single-item create/read/update for students and a full list for administrators; no joins are required.	Table-level, from the EC2 instance role.	`PutItem`, `GetItem`, `Scan` and `UpdateItem` granted; no `DeleteItem`; admin list uses `Scan` (O(table)); no `Query` because there is no index.
Capacity	On-demand capacity matches unpredictable pilot traffic and avoids provisioned-capacity planning.	Table-level in us-east-1.	On-demand; warm throughput 12,000 read / 4,000 write units per second; no auto-scaling activities.
Resilience	PITR and backups would protect against accidental writes or deletes; both are currently off.	Table-level in us-east-1.	PITR Off; no manual backups; Streams Off; deletion protection Off; TTL Off.
Encryption and governance	Encrypts data at rest with no key-management overhead for the pilot.	Table-level in us-east-1.	AWS-owned key; resource-based policy not active; no tags.
Relationship to S3	The table is the system of record; the S3 prefix uses the same `complaintId`, so evidence can be located without a second identifier.	Same region as the bucket and the instance.	No cross-region replication; object keys are derived from the table key by the application.

Design observations

- Single-table, single-attribute access pattern: everything is keyed on the complaint UUID; admin listings rely on `Scan` - fine at this scale, but it is an O(table) read that grows with data.
- Backup posture: PITR and Streams are off and no manual backups exist; only console-created operations would protect it.
- `ProcessedPartnerEvents` (same region, part key `eventId`) is **NOT part of this project** - it belongs to an earlier event-processing lab.

4.6. Identity (Cognito)

Component	Why it is used	Placement	Configuration and security
User pool	Central identity store for students and administrators; removes password storage and sign-in logic from the application.	`us-east-1_5MCukO9qb` (`User pool - f-p6pz`) in us-east-1; OIDC discovery at `https://cognito-idp.us-east-1.amazonaws.com/us-east-1_5MCukO9qb/.well-known/openid-configuration`.	Essentials plan; 3 confirmed users; email sign-in; self-registration enabled with e-mail verification; password policy 8+ characters with upper, lower, number and special character; MFA off; account recovery by e-mail then SMS.
App client	Public single-page-application client that runs the OIDC authorization-code flow.	`CollegeComplaintApp`; no client secret (the client ID is listed in Appendix A).	Authorization code grant; scopes `email openid phone profile`; callback `https://ip-10-0-1-99.taild430d2.ts.net/auth/callback`; sign-out `https://ip-10-0-1-99.taild430d2.ts.net`; access and ID tokens 60 minutes, refresh token 5 days; token revocation and prevent-user-existence errors enabled.
Hosted UI	Provides a managed sign-in and sign-up page, so the team does not build and maintain an authentication UI.	Cognito prefix domain `https://us-east-15mcuko9qb.auth.us-east-1.amazoncognito.com`; no custom domain.	Hosted UI (classic) branding; managed login available; TLS terminated by Cognito; no ACM certificate or custom DNS record.

Component	Why it is used	Placement	Configuration and security
Groups and roles	Separates administrators from students without adding a second user store.	User pool `Admins` group (description: CampusCare administrators).	One group; no precedence; group membership is exposed in the token for the backend to enforce administrator actions.
JWT validation	The Flask API verifies tokens locally against the pool's public keys, so it does not call Cognito on every request.	Backend on EC2; JWKS at `https://cognito-idp.us-east-1.amazonaws.com/us-east-1_5MCukO9qb/.well-known/jwks.json`.	RS256 signature check; issuer and audience validation in the application; no identity pool, so the browser never receives AWS credentials.
Federation and MFA	No social/SAML federation or MFA in the pilot; both are available as next steps.	User-pool level in us-east-1.	Identity providers: Cognito user pool directory only; MFA enforcement No MFA; device tracking off; passkey user verification Preferred.

Design observations

- **User-pool-only** auth with the hosted UI: the React client uses `oidc-client-ts` to run the authorization-code flow against `CollegeComplaintApp`; the Flask API validates the resulting JWTs against the pool's JWKS. There is no identity pool, so the browser never receives AWS credentials.
- The hosted UI domain is configured (`us-east-15mcuko9qb.auth.us-east-1.amazoncognito.com`) but there is no custom domain or ACM certificate. The `Admins` group is the only role boundary; students and administrators share the same pool.

4.7. Messaging (SNS)

Component	Why it is used	Placement	Configuration and security
Topic	Decouples status-change notifications from the request path; one publish fans out to every subscriber.	`CollegeComplaintNotifications` in us-east-1 (`arn:aws:sns:us-east-1:547846135503:CollegeComplaintNotifications`); called through the public SNS endpoint - no VPC endpoint is provisioned for SNS.	Standard topic; 256 KiB maximum message size; no display name; encryption disabled.
Subscriptions	Delivers events to a durable queue for downstream processing and to the administrator's e-mail for visibility.	SQS queue `CollegeComplaint_Queue` and e-mail `justhalfy@gmail.com`, both in us-east-1.	Both subscriptions Confirmed; SQS gives durable fan-out, e-mail is used for demonstration and administrator alerts.
Access control	Only the application role can publish; the app cannot add or remove subscribers.	Topic policy in us-east-1.	Default topic policy with `Principal: *` restricted by `AWS:SourceAccount = 547846135503`; the IAM role grants only `sns:Publish`; no `Subscribe` or topic-administration grants.
Delivery	Standard topics provide at-least-once delivery and retry behaviour without managing brokers.	Regional managed service in us-east-1.	At-least-once delivery; no encryption; no HTTP/S delivery policy.

Design observations

- The topic is a standard topic with two confirmed subscriptions: an SQS queue for durable downstream processing and an administrator e-mail address for visibility. The application can only publish; subscription management remains an administrative action.
- SNS encryption is disabled. Enabling it would require a KMS key whose policy allows the topic to use the key, which is a reasonable next step once the notification payload contains sensitive complaint data.

4.8. Observability (CloudWatch)

Component	Why it is used	Placement	Configuration and security
EC2 metrics	Provides free basic health and performance signals for the single instance.	CloudWatch us-east-1, namespace `AWS/EC2`, dimension `InstanceId=i-0789decbb3db6fa77`.	Basic 5-minute metrics; detailed monitoring disabled; no custom metrics.
Application logs	Central place for Flask and Nginx logs; the instance role can write without static keys.	Log group `CollegeComplaintApp` in us-east-1; the instance calls the public CloudWatch Logs endpoint (no VPC endpoint).	Log group exists; Standard log class; retention 1 week; deletion protection Off; no data-protection or anomaly detection; no metric filters or subscription filters.
Alarms	Would alert operators when the instance or application degrades; no project alarm is configured.	CloudWatch Alarms in us-east-1.	0 project alarms; 2 legacy lab alarms (`DPT-CW-ALARM-EX21-10639`, `CloudLab7-Cognito-SignUp-Alarm`) are in INSUFFICIENT_DATA.
Dashboards	Would give a single operational view of the workload; none exists for CampusCare.	CloudWatch dashboards in us-east-1.	No CampusCare dashboard; the existing dashboards belong to earlier labs.
Logging path	The application can write logs through the instance role, with the log group as the only permitted destination.	EC2 -> CloudWatch Logs public endpoint -> `CollegeComplaintApp`.	IAM grants `CreateLogStream`, `PutLogEvents` and `DescribeLogStreams`; no `CreateLogGroup` or `DeleteLogGroup`; retention 1 week.

4.9. Service summary

The table below consolidates the services described above and their role in the application.

Component	Why it is used	Placement	Configuration and security
Cognito	Authentication, token issuance and JWT verification.	Regional user pool `us-east-1_5MCukO9qb` and the hosted UI domain.	Authorization-code flow, 60-minute access/ID tokens, MFA off.
EC2	Application compute: Nginx, React UI and Flask API.	Public subnet `10.0.1.0/24`, us-east-1a.	`t3.micro`, IMDSv2 required, security group 80/443 plus admin SSH.
DynamoDB	Complaint metadata persistence.	Regional table `CollegeComplaints`; DynamoDB gateway endpoint.	On-demand capacity, partition key `complaintId`, PITR off.
S3	Private evidence storage, one prefix per complaint.	Regional bucket; S3 gateway endpoint.	SSE-S3, Block Public Access on, versioning off.
SNS	Status notification fan-out.	Regional standard topic.	Two confirmed subscriptions, encryption off.
IAM	Least-privilege access for the instance.	Instance profile attached to EC2.	Four scoped policies, no static access keys.
VPC	Network isolation, private database tier and gateway endpoints.	`10.0.0.0/16`, two public and two private subnets.	Internet gateway, no NAT gateway, S3/DynamoDB gateway endpoints.
CloudWatch	Basic metrics and application logs.	Regional log group `CollegeComplaintApp`.	5-minute metrics, one-week log retention, no project alarms.

5. Request Flow (complaint lifecycle)

The sequence diagram in Figure 2 shows the full request lifecycle across the user, Amazon Cognito, the EC2 backend and the AWS services. It covers the sign-up / sign-in phase (steps 1-8) and the administrator phase (steps 9-10).

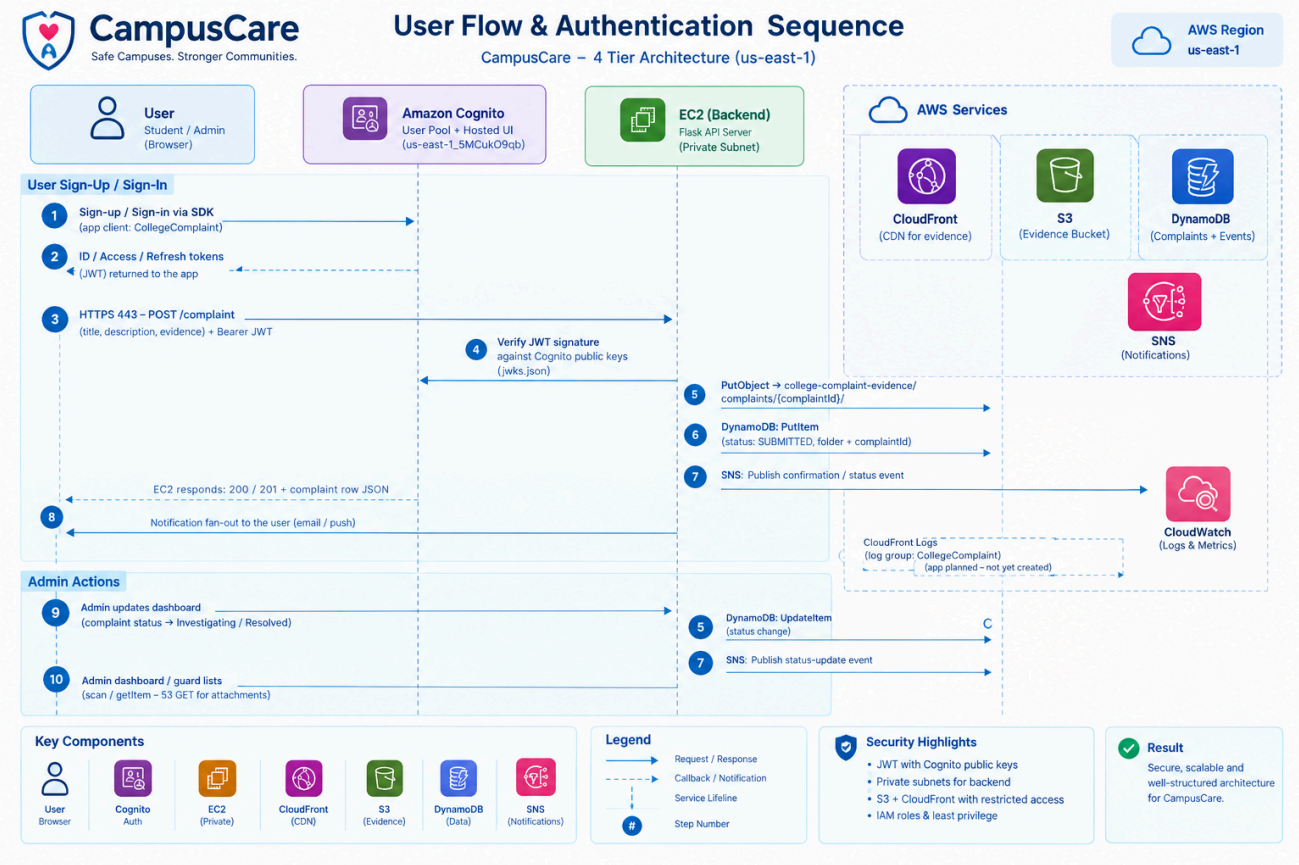


Figure 2: CampusCare - user flow and authentication sequence. All AWS calls use the instance role `CollegeComplaints-EC2-Role`; no static credentials are stored on the instance.

The lifecycle proceeds as follows:

1. **Sign-in.** The student opens the Cognito hosted login page through the `CollegeComplaintApp` app client, completes the authorization-code flow, and receives ID, access and refresh tokens (JWT).
2. **Authorised request.** The browser sends the access token as a `Bearer` credential with every API request over HTTPS. Tailscale Funnel terminates TLS on the instance and forwards the request through Nginx to the Flask API.
3. **Token validation.** Flask validates the JWT signature against the user pool's JWKS endpoint and extracts the user identity and role.
4. **Evidence upload.** For a new complaint, Flask generates a complaint UUID and uploads each attachment to `s3://college-complaint-evidence/complaints/<uuid>/` using `s3:PutObject`.
5. **Metadata write.** Flask writes the complaint record - owner, title, description, status `SUBMITTED`, evidence prefix and timestamp - to the `CollegeComplaints` table with `dynamodb:PutItem`.
6. **Confirmation event.** Flask publishes a confirmation event to `CollegeComplaintNotifications` with `sns:Publish`.
7. **Administrator review.** An administrator lists complaints with `dynamodb:Scan`, opens one with `dynamodb:GetItem`, and retrieves attachments from S3 with `s3:GetObject`.
8. **Status update.** The administrator changes the status (for example `SUBMITTED -> UNDER_REVIEW -> RESOLVED`) with `dynamodb:UpdateItem`, and a status event is published to SNS.
9. **Notification.** Subscribers receive the status change by e-mail or SMS, and the student sees the updated status on the dashboard.
10. **Logging.** Application logs are written to the CloudWatch log group `CollegeComplaintApp` (one-week retention); the group exists, but no alarm or dashboard is configured.

6. Application Walkthrough

The screenshots in this chapter were captured from the live deployment on November 1, 2026 and follow the same lifecycle described in Section 5. They show the React single-page application, the Amazon Cognito hosted sign-in page, the evidence stored in Amazon S3 and the e-mail notification delivered by Amazon SNS. The records visible in the screenshots are test data created while validating the deployment.

6.1. Student experience

A student first signs in through the Cognito hosted UI, lands on the dashboard, submits a complaint with an evidence photo, and can follow its status and notifications from the same portal.

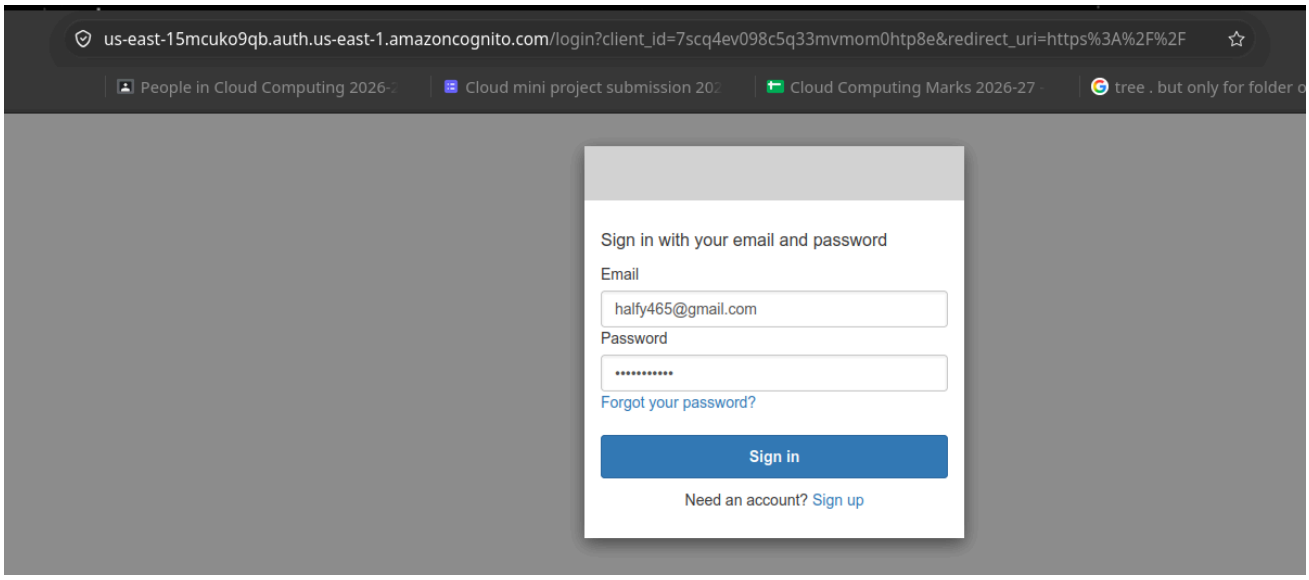


Figure 3: Cognito hosted UI sign-in page. The browser is redirected to the user-pool domain `us-east-15mcuko9qb.auth.us-east-1.amazoncognito.com` with the `CollegeComplaintApp` client ID and the authorization-code callback.

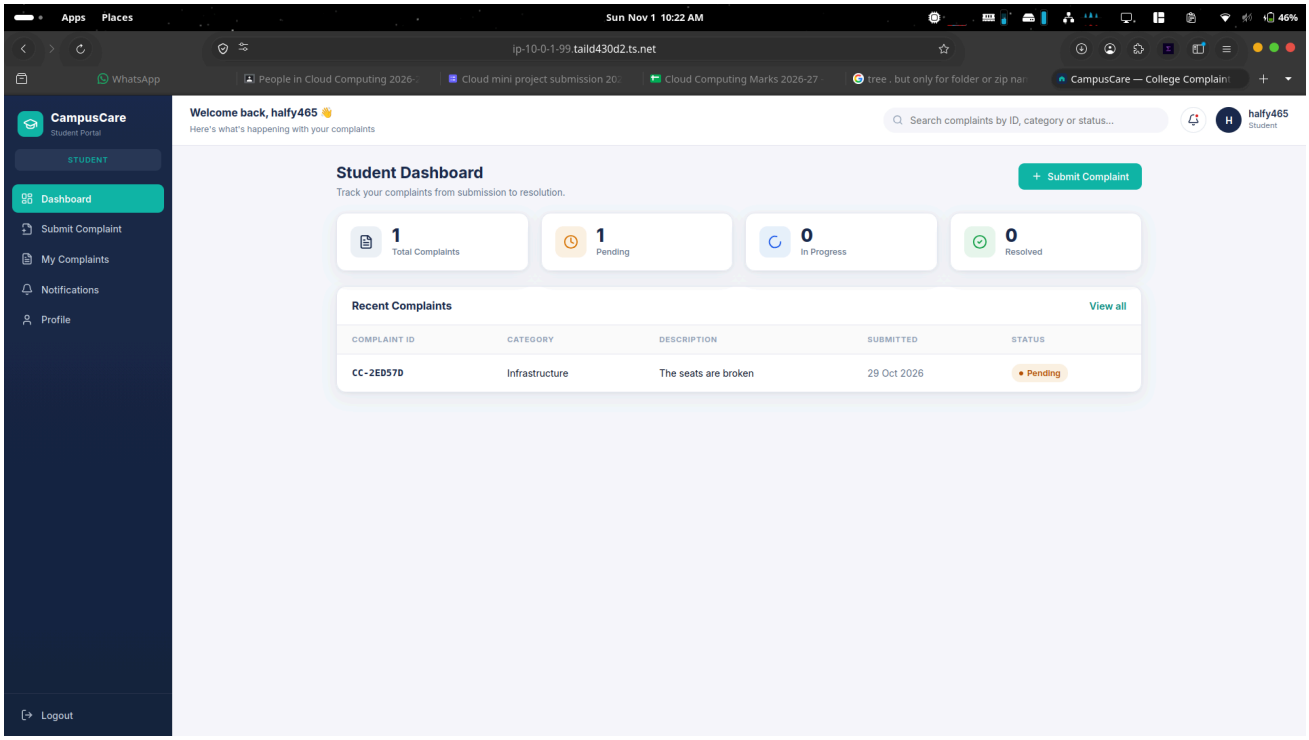


Figure 4: Student dashboard after sign-in, showing complaint counters and the most recent complaint for the signed-in student.

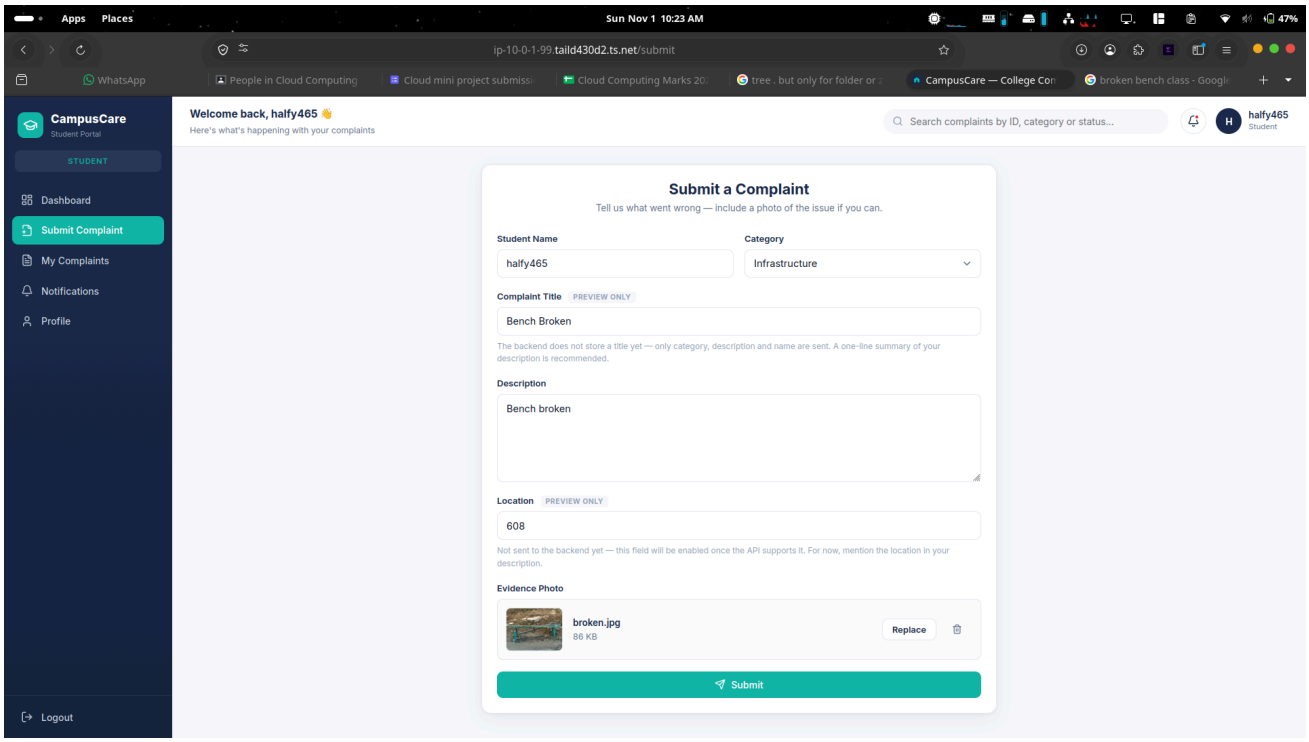


Figure 5: Complaint submission form. The category, description and evidence file are sent to the Flask API; the title and location fields are preview-only in this version.

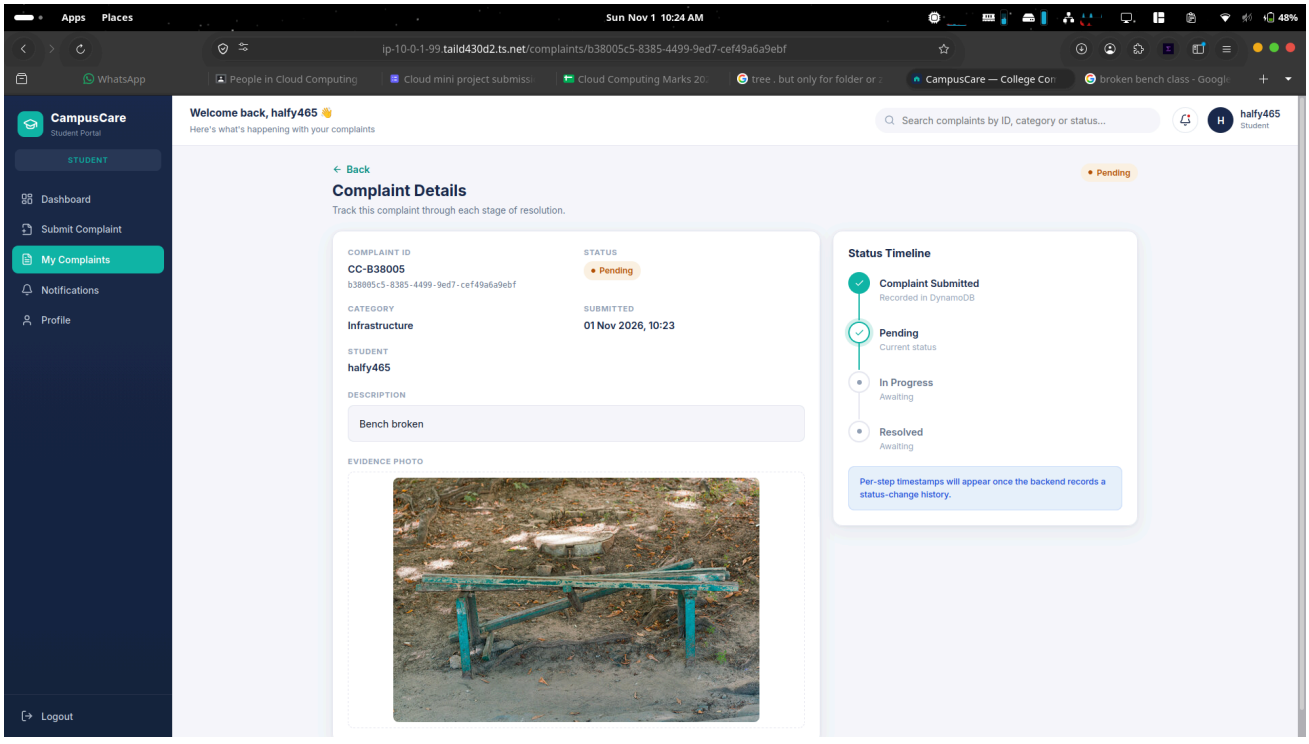


Figure 6: Student complaint detail view. The evidence photograph is streamed from the private S3 bucket through the authenticated API, and the status timeline reflects the DynamoDB record.

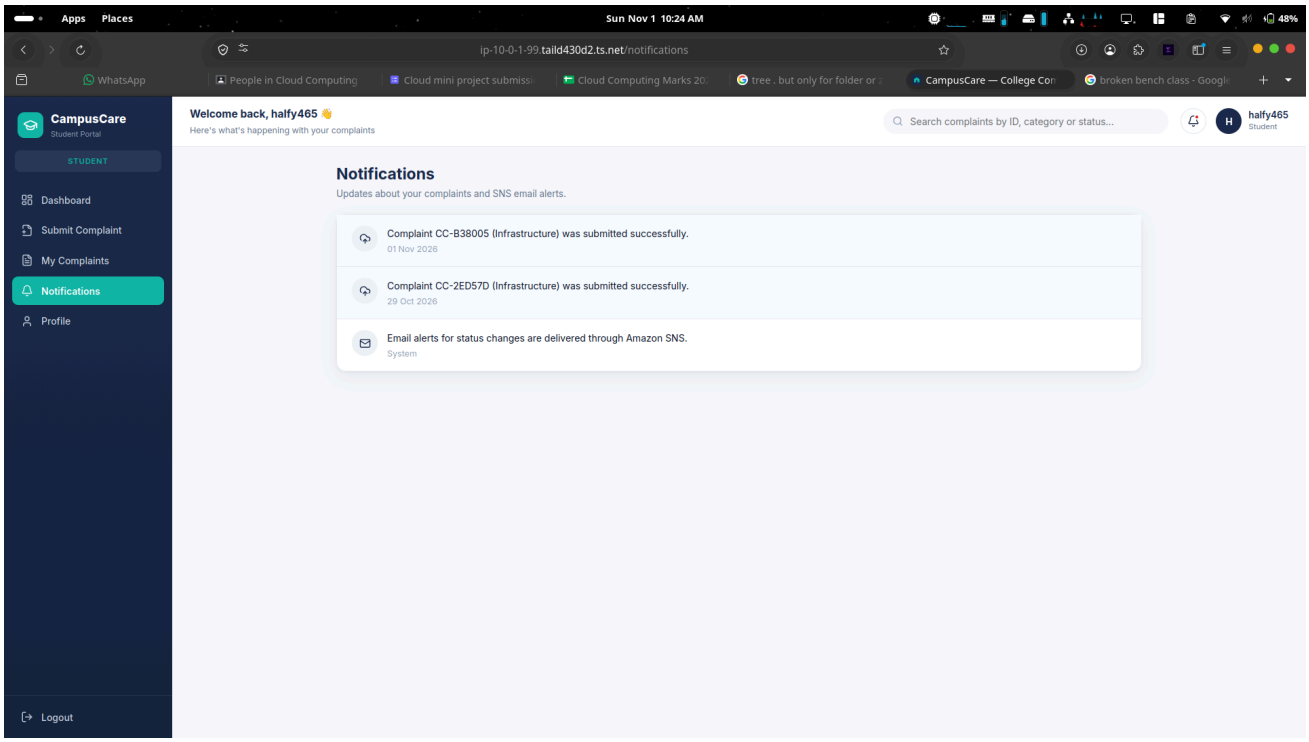


Figure 7: Student notifications page. Submission and status events are listed in the application and mirrored to the SNS e-mail subscription.

6.2. Administrator experience

Administrators use a separate console view backed by the same DynamoDB table and S3 bucket. The dashboard summarises the workload, while the complaint list and review dialog expose the evidence and the status controls.

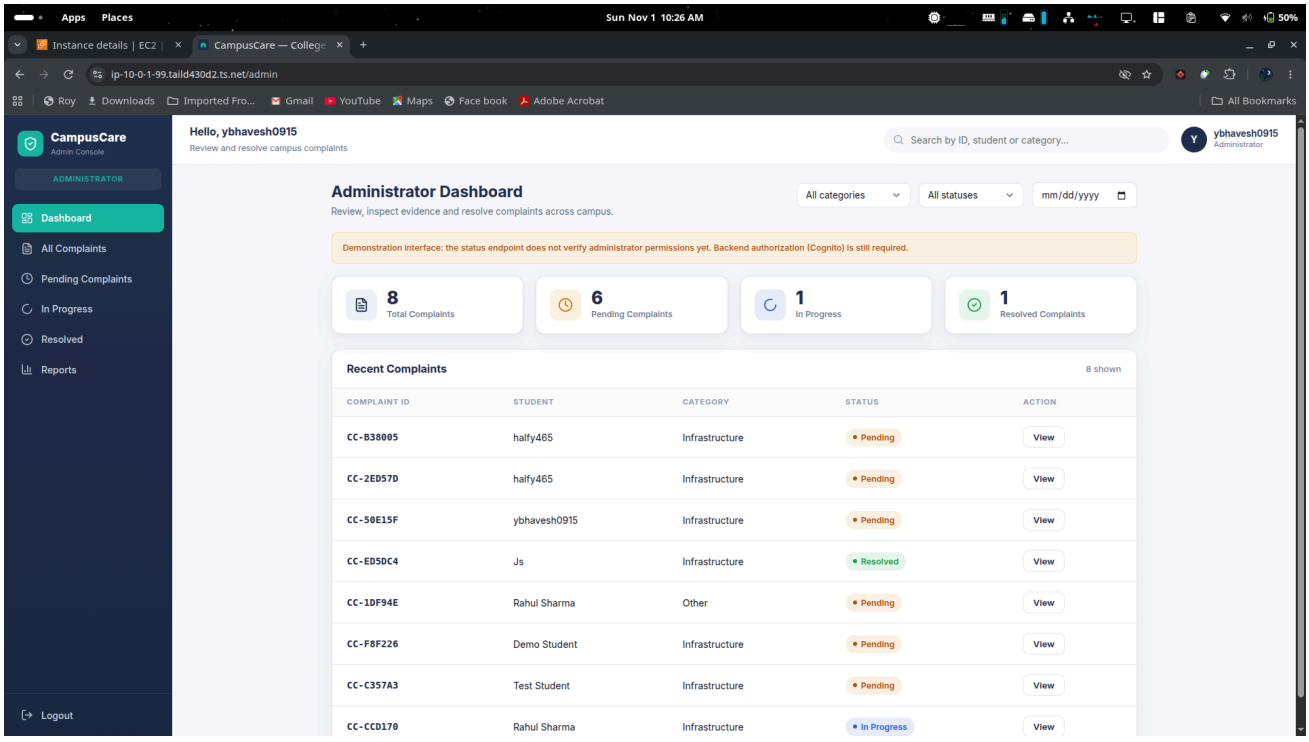


Figure 8: Administrator dashboard with complaint counts by status and a recent-complaints table.

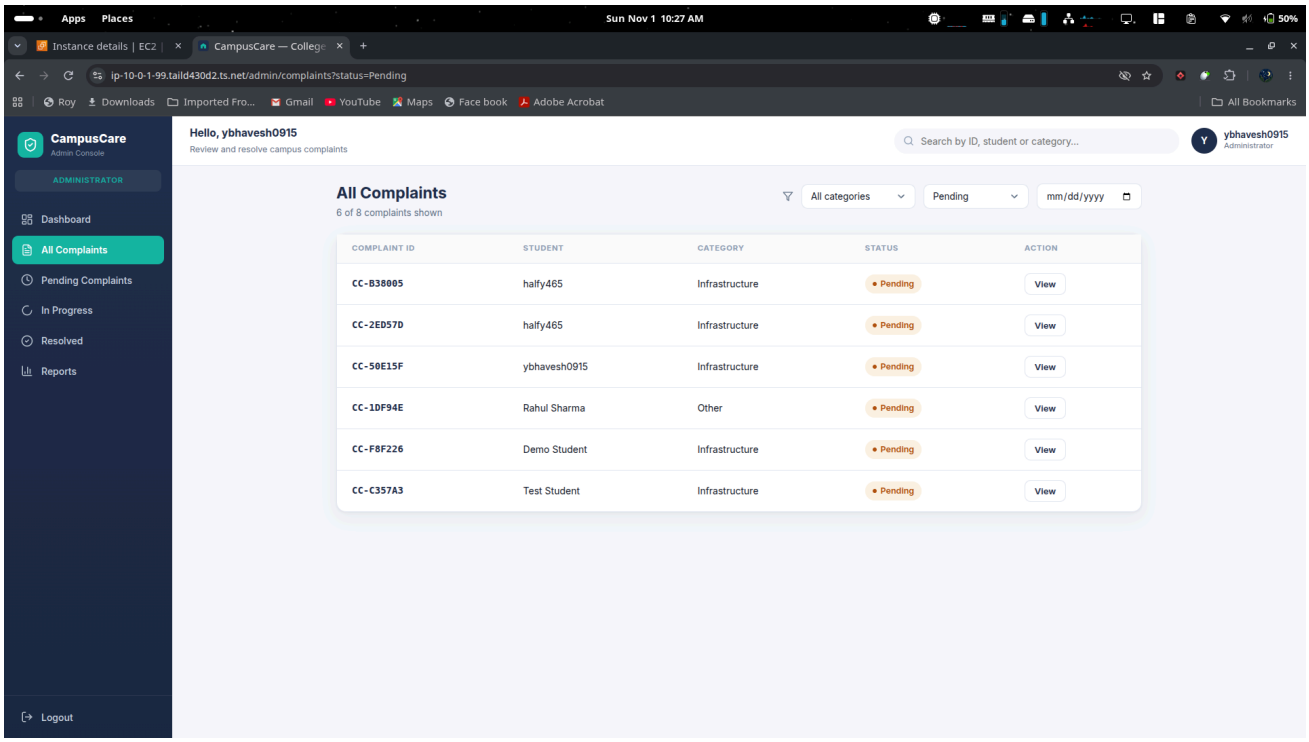


Figure 9: Administrator complaint list with category and status filters. The list is produced by a DynamoDB `Scan` in this version.

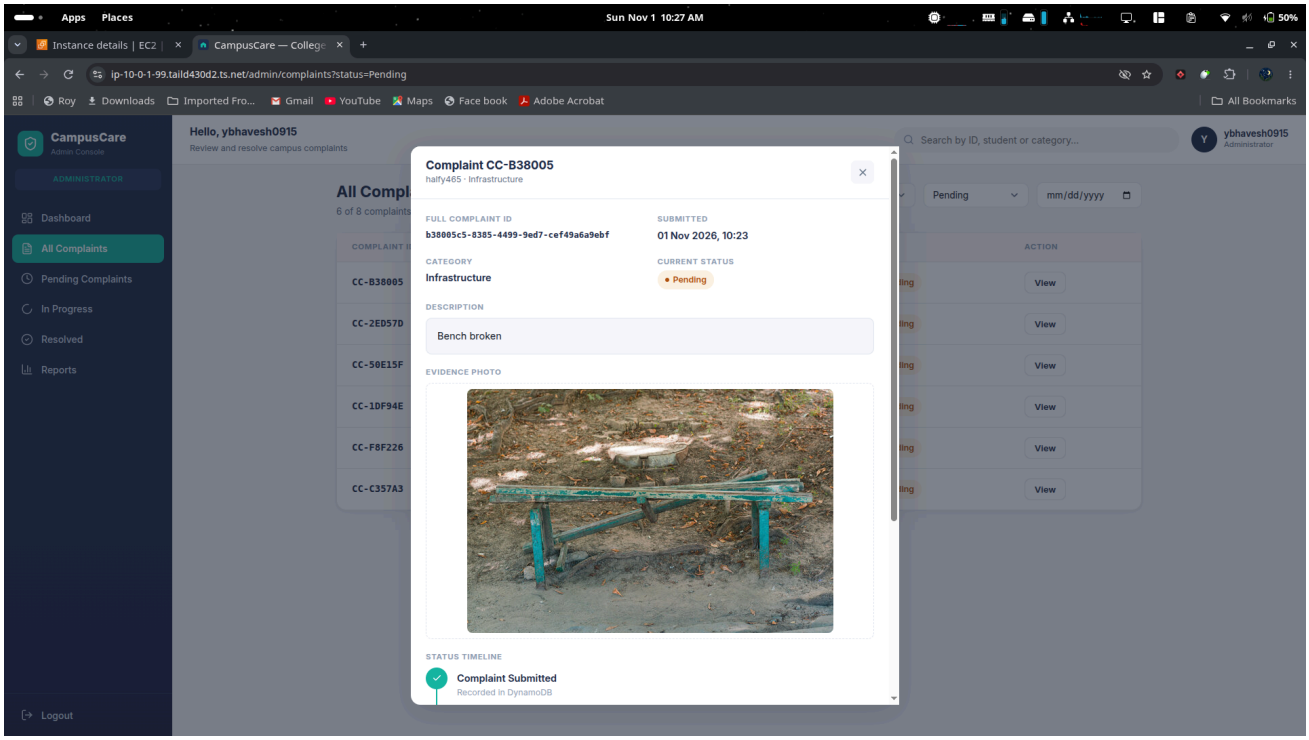


Figure 10: Administrator review dialog showing the complaint metadata, the S3-hosted evidence and the status timeline.

6.3. Notification delivery

Each submission and status change is published to Amazon SNS. The topic's e-mail subscription delivers the message shown below to the administrator, and the SQS subscription provides a durable copy for downstream processing.

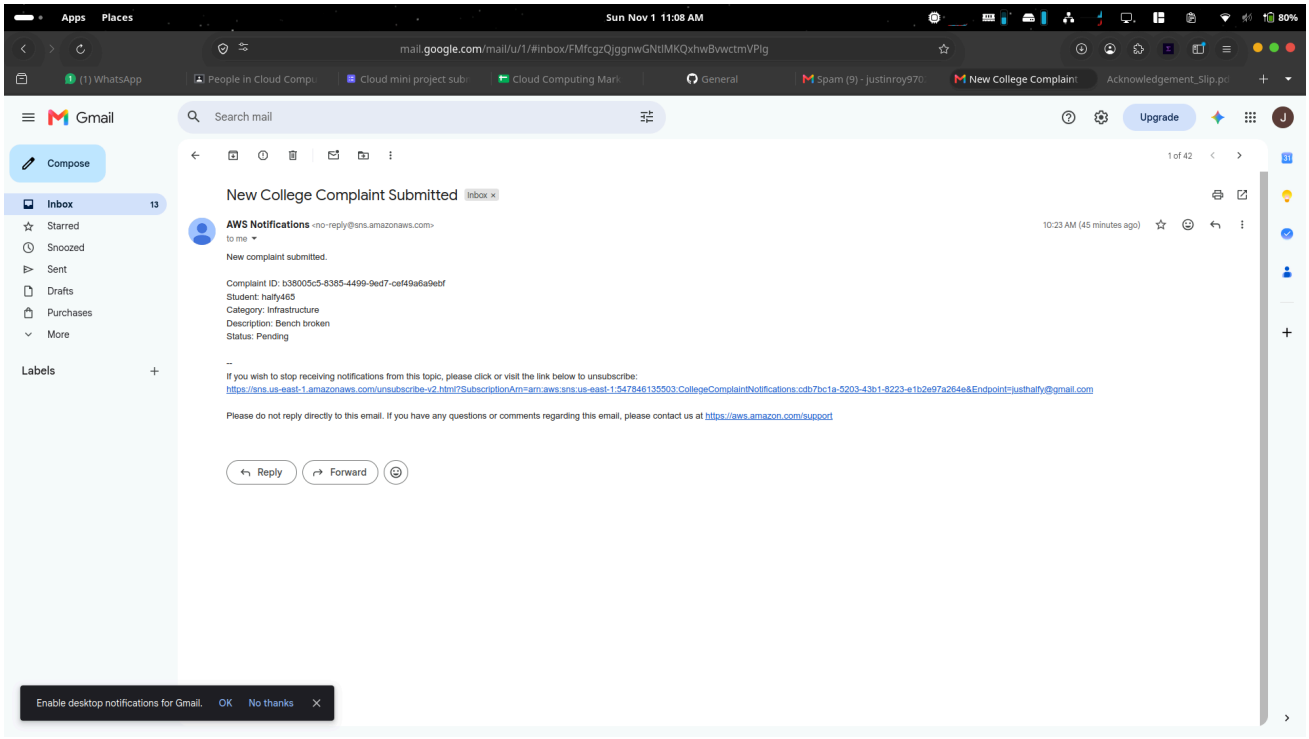


Figure 11: E-mail delivered by the Amazon SNS e-mail subscription after a complaint is submitted. The message contains the complaint ID, student, category, description and status.

7. Conclusion and Future Work

CampusCare demonstrates that a complete complaint-management workflow can run on a minimal AWS footprint. A single `t3.micro` instance, a Cognito user pool, one DynamoDB table, one private S3 bucket and one SNS topic provide authentication, persistence, evidence storage and notification at a cost close to zero under the Free Plan. The use of an IAM instance role with four narrowly scoped policies keeps credentials off the server, while the VPC design isolates the application from the future database tier and routes S3/DynamoDB traffic through free gateway endpoints.

Key outcomes

- A working end-to-end complaint workflow hosted on a single EC2 instance.
- Least-privilege AWS access through an IAM instance role, with no static credentials on the server.
- A VPC baseline (public application subnet, private database subnets and gateway endpoints) ready for the next stage.
- A documented service inventory that records, for each component, why it is used, where it is placed and how it is configured.

The current version is a functional pilot. The next stage should harden operations (logging, alarms, backups, encryption), improve the data access pattern (GSI + `Query`), and make the deployment resilient (multi-AZ, Auto Scaling, infrastructure as code). Longer-term work includes a native mobile client, integration with the college ERP for student records, an analytics dashboard for complaint trends, and role-based workflows for departments and escalation.

8. References

1. Amazon Web Services. *Amazon EC2 User Guide for Linux Instances.* docs.aws.amazon.com/AWSEC2/latest/UserGuide/
2. Amazon Web Services. *Amazon Virtual Private Cloud User Guide.* docs.aws.amazon.com/vpc/latest/userguide/
3. Amazon Web Services. *Amazon Cognito Developer Guide.* docs.aws.amazon.com/cognito/latest/developerguide/
4. Amazon Web Services. *Amazon DynamoDB Developer Guide.* docs.aws.amazon.com/amazondynamodb/latest/developerguide/
5. Amazon Web Services. *Amazon Simple Storage Service User Guide.* docs.aws.amazon.com/AmazonS3/latest/userguide/
6. Amazon Web Services. *Amazon Simple Notification Service Developer Guide.* docs.aws.amazon.com/sns/latest/dg/
7. Amazon Web Services. *AWS Identity and Access Management User Guide.* docs.aws.amazon.com/IAM/latest/UserGuide/
8. Amazon Web Services. *Amazon CloudWatch User Guide.* docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/
9. Tailscale Inc. *Tailscale Funnel.* tailscale.com/kb/1223/funnel
10. Nginx. *Nginx Documentation.* nginx.org/en/docs/
11. Pallets Projects. *Flask Documentation.* flask.palletsprojects.com
12. Meta Open Source. *React Documentation.* react.dev

Appendix A Resource-ID Quick Reference

```
EC2 instance         i-0789decbb3db6fa77          (CollegeComplaint-Server)  Running  t3.micro
EBS root volume      vol-0d5b3d830cde5914a        gp3 8GiB 3000 IOPS
Key pair             CollegeComplaint
AMI                  ami-0d97d7f07b25dd693        Ubuntu 26.04 Server amd64 (gp3)
Public IPv4          3.238.21.215                 (auto-assigned)
Private IPv4         10.0.1.99
VPC                  vpc-0e9087bb4d0c51f1e        CollegeComplaint-VPC       10.0.0.0/16
Internet Gateway     igw-068e33895a1b1eea6        CollegeComplaint-igw
Public subnets       subnet-075715342703e45c8     CollegeComplaint_public    10.0.1.0/24 us-east-1a
                     subnet-003423eca685068d2     CollegeComplaint_public_2  10.0.2.0/24 us-east-1b (empty)
Private subnets      subnet-05e13771ef6f4f787     CollegeComplaint_private   10.0.3.0/24 us-east-1a (empty)
                     subnet-0aa598fe5df874443     CollegeComplaint_private_2 10.0.4.0/24 us-east-1b (empty)
Route Tables         rtb-0d1f57e36d0516e27        public RT   0.0.0.0/0 -> IGW
                     rtb-092817e4a5469aa36        private RT  no internet route (prefix-lists only)
Gateway Endpoints    vpce-0e07cb8333f836314       CC-S3-endpoint (S3)        both RTs
                     vpce-0f0022dd2895aceb1       CC-DDB-endpoint (DDB)      both RTs
Prefix lists         pl-63a5400a (S3) / pl-02cd2c6b (DynamoDB)  route entries in both RTs
DB Subnet Group      collegecomplaint-db-subnet-group     both private subnets (Complete)
Security Group       sg-045711f296c742039         CollegeComplaint-SG        80,443 open · 22 admin-IP
CloudFront           (pending account verification)  S3 origin + OAC wizard prepared
IAM Role             CollegeComplaints-EC2-Role   instance profile attached
IAM Policies         CollegeComplaints-S3-Policy  CollegeComplaints-DynamoDB-Policy
                     CollegeComplaints-SNS-Policy CollegeComplaints-CloudWatch-Policy
S3 Bucket            college-complaint-evidence   complaints/{uuid}/ SSE-S3 BPBA ON
DynamoDB Table       CollegeComplaints            PK complaintId(String) on-demand
Cognito User Pool    us-east-1_5MCukO9qb          CollegeComplaintApp client 7scq4ev098c5q33mvmom0htp8e
Cognito domain       us-east-15mcuko9qb.auth.us-east-1.amazoncognito.com
SNS Topic            CollegeComplaintNotifications (Standard)  2 confirmed subscriptions
CW Log group         CollegeComplaintApp          Standard, 1-week retention
```